Pashov Audit Group

Regnum Aurum Security Review

March 7th 2026 - March 8th 2026

Contents

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Regnum Aurum

Regnum Aurum (RAAC) is a fractionalization platform that tokenizes real estate into NFTs (RAACNFT) and fractional index tokens (iRAAC), enabling on-chain lending, borrowing, and liquidity against property value. By combining Chainlink-powered appraisals, a hybrid RWA Vault, and veRAAC governance the protocol enables programmable debt positions against real estate assets with on-chain liquidation mechanisms.

5. Executive Summary

A time-boxed security review of the **RegnumAurumAcquisitionCorp/contracts** repository was done by Pashov Audit Group, during which **Blockace**, **Drynooo**, **Said** engaged to review **Regnum Aurum**. A total of **2** issues were uncovered.

Protocol Summary	
Project Name	Regnum Aurum
Protocol Type	RWA tokenization
Timeline	March 7th 2026 - March 8th 2026

Review commit hash:
- 4f940d02dc8ff1660c56cc9cb6f2b74a25e81c1e (RegnumAurumAcquisitionCorp/contracts)

Fixes review commit hash:
- e15afd14e454277d600afd1538d29d0a4ca053bd (RegnumAurumAcquisitionCorp/contracts)

Scope

`SToken.sol` `ISToken.sol`

6. Findings

Findings count

Severity	Amount
Low	2
Total findings	**2**

Summary of findings

ID	Title	Severity	Status
[L-01]	`withdraw` has zero effective slippage protection and no FOT handling	Low	Resolved
[L-02]	Missing compliance check on receiver in transferWithdrawRights	Low	Resolved

Low findings

[L-01] `withdraw` has zero effective slippage protection and no FOT handling

Description

The contract claims to support fee-on-transfer tokens. The deposit side correctly handles FOT via balance-before-after measurement and provides functional `minAmountOut` slippage protection. The withdrawal side has neither.

`maxAmountBurned` is dead code, `previewWithdraw()` is a function that always returns `amountOut` unchanged. Therefore, `amountBurned == amountOut` always.

```
// ...
uint256 amountBurned = previewWithdraw(token, amountOut);     // always returns amountOut
if (amountBurned > maxAmountBurned) revert SlippageExceeded(); // never triggers in practice
// ...
```

And there is no `minReceived` for FOT tokens. For FOT tokens, `safeTransfer` delivers `amountOut - fot_fee` to the user. The user has no parameter to enforce a minimum received amount.

Consider adding a `minReceived` parameter.

[L-02] Missing compliance check on receiver in transferWithdrawRights

Description

`transferWithdrawRights()` only calls `_checkCompliance(msg.sender)`, but does not check compliance on the `to` parameter. This is inconsistent with `_update()`, which checks compliance on both `from` and `to` for token transfers.

```
function transferWithdrawRights(address token, address to, uint256 amount) external
nonReentrant onlyRole(MANAGER_ROLE) {
    if (paused() && !emergencyMode) revert EnforcedPause();
    _checkCompliance(msg.sender); // @audit - only checks sender
    if (token == address(0) || to == address(0)) revert InvalidAddress();
    if (amount == 0) revert ZeroAmount();
    if (userTokenDeposits[msg.sender][token] < amount) revert InsufficientDeposit();
    userTokenDeposits[msg.sender][token] -= amount;
    userTokenDeposits[to][token] += amount;
}
```

Consider adding a compliance check on the `to` address:

```
function transferWithdrawRights(address token, address to, uint256 amount) external
nonReentrant onlyRole(MANAGER_ROLE) {
    if (paused() && !emergencyMode) revert EnforcedPause();
    _checkCompliance(msg.sender);
+   _checkCompliance(to);
    if (token == address(0) || to == address(0)) revert InvalidAddress();
    ...
}
```